Principles of Compiler Design Lecture & Lab

CST-405 4 Credits Sep 8 - Dec 20

Course Description

This course reviews the concepts and tools used in the development of compilers. Students synthesize topics covered in previous courses: formal languages, data structures, and computer architecture. The course reinforces the principles of software engineering and development through a complete cycle of building a working compiler. The laboratory reinforces and expands learning of principles introduced in the lecture. Hands-on activities focus on writing a compiler including a lexer, parser, semantic analyzer, code generator, and optimizer. Prerequisites: CST-301 and MAT-374.

Instructor Contact Information

Nasser Tadayon
Nasser.Tadayon@gcu.edu

Isac Artzi
Isac.Artzi@gcu.edu

Class Resources

Compilers: Principles, Techniques, & Tools (REQUIRED)

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, techniques, & tools* (2nd ed.). Addison-Wesley. ISBN-13: 9780321486813.

Optional - Stack Overflow (OPTIONAL)

For additional information, the following is recommended:

Stack Overflow is site for programmers that provides a library of detailed answers to various questions about programming.

https://stackoverflow.com/

Visual Studio Code (REQUIRED)

This course requires the use of Visual Studio Code.

Be sure to download or update to the version specified by your instructor.

https://code.visualstudio.com/download

QTSPIM: MIPS Assembly Simulator (for MAC users) (REQUIRED)

This course requires the use of QTSPIM: MIPS Assembly Simulator.

Access to and additional information for resources may be found on the Course Materials Support page in the Student Success Center.

Be sure to download or update to the version specified by your instructor.

http://www.gcumedia.com/digital-resources/softpedia/2017/qt-spim_mips-assembly-simulator_macos_1e.php

QTSPIM: MIPS Assembly Simulator (for Window users) (REQUIRED)

This course requires the use of QTSPIM: MIPS Assembly Simulator.

Access to and additional information for resources may be found on the Course Materials Support page in the Student Success Center.

Be sure to download or update to the version specified by your instructor.

http://www.gcumedia.com/digital-resources/informer-technologies/2017/qt-spim_mips-assembly-simulator_windowsos_1e.php

GIT Source Code Repositories ()

This course requires the use of GIT Source Code Repositories (GitHub).

Access to and additional information for resources may be found on the Course Materials Support page in the Student Success Center.

Be sure to download or update to the version of the GIT Client specified by your instructor.

Compiler Design Tutorial (REQUIRED)

Refer to this resource as needed.

https://www.tutorialspoint.com/compiler_design/index.htm

Phases of a Compiler (REQUIRED)

Read "Phases of a Compiler," by Jha, from Geeks for Geeks (2021).

https://www.geeksforgeeks.org/phases-of-a-compiler/

What is a Compiler Design? Types, Construction Tools, Example (REQUIRED)

Read "What is a Compiler Design? Types, Construction Tools, Example," by Smith, from Guru99 (2022).

https://www.guru99.com/compiler-design-tutorial.html

CST-405 Compiler Design Checklist (REQUIRED)

Refer to this resource for select assignments in the course.

CST-405-CompilerDesignChecklist.docx

CST-405 Course Guide (Instructor Only)

CST-405-CourseGuide-Instructor.docx

CST-405 Course Revision History (Instructor Only)

CST-405-CRH-ONGROUND.pdf

CST-405 Discussion Forum for Instructors (Instructor Only)

CST-405-D-ONGROUND-Instructor.docx

Topic 1: Compiler Design Phases

Sep 8, 2026 - Sep 13, 2026 Max Points: 110

It is necessary within the field to have experience in the design and implementation of programming language compilers. To be successful in this endeavor, best practice dictates a full understanding of all phases of compiler design prior to implementing them.

Objectives:

1. Examine the theoretical principles, design practices, and implementation strategies of compiler design.
2. Complete a lexical analyzer.
3. Configure a compiler development environment.

Resources

Compilers: Principles, Techniques, & Tools (REQUIRED)

Read Chapters 1–3 and browse through Chapters 4–8 in *Compilers: Principles, Techniques, & Tools*.

Assessments

Summary of Current Course Content Knowledge

Start Date & Time	Due Date & Time	Points
Sep 8, 2026, 12:00 AM	Sep 8, 2026, 11:59 PM	0

Assessment Description

Academic engagement through active participation in instructional activities related to the course objectives is paramount to your success in this course and future courses. Through interaction with your instructor and classmates, you will explore the course material and be provided with the best opportunity for objective and competency mastery. To begin this class, review the course objectives for each Topic, and then answer the following questions as this will help guide your instructor for course instruction.

1. Which weekly objectives do you have prior knowledge of and to what extent?
2. Which weekly objectives do you have no prior knowledge of?
3. What course-related topics would you like to discuss with your instructor and classmates? What questions or concerns do you have about this course?

Class Introductions

Start Date & Time	Due Date & Time	Points
Sep 8, 2026, 12:00 AM	Sep 9, 2026, 11:59 PM	0

Assessment Description

Take a moment to explore your new classroom and introduce yourself to your fellow classmates. What are you excited about learning? What do you think will be most challenging?

Lab Question 1

Start Date & Time	Due Date & Time	Points
Sep 8, 2026, 12:00 AM	Sep 9, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

Explain the role of a lexical analysis. Provide concrete examples of two code snippets from two different programming languages and show two different tokens returned for each code snippet.

Lab Question 2

Start Date & Time	Due Date & Time	Points
Sep 8, 2026, 12:00 AM	Sep 11, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

What is the difference between creating a lexical analyzer with a tool like Flex and building one directly, for example in C or C++? Substantiate your explanation with concrete code examples.

CLC – Project 1: Lexical Analyzer

Start Date & Time	Due Date & Time	Points
Sep 8, 2026, 12:00 AM	Sep 13, 2026, 11:59 PM	90

Assessment Traits

Group

Assessment Description

This is a Collaborative Learning Community (CLC) assignment.

In this project, students will resubmit the lexical analyzer that was constructed in CST-301. This will act as the basis for other projects done in this course.

In a Microsoft Word document, complete and submit the following:

1. Evidence of access to Unix/Linux.
2. Evidence of proper installation of Flex and Bison.

Make sure all screenshots demonstrate the execution and explanation on what is being depicted.

Then, complete the lexical analyzer (from CST-301) with the following functionality:

1. Recognizes every valid keyword, expression, or construct defined in the grammar provided.
2. Returns the correct token and its kind.
3. Generates lexical errors when there are errors in the code, including their exact location.
4. Certifies that the source code is correct when there are no lexical errors.

Additionally, submit:

1. All code to your private GitHub account, making sure to provide extensive comments in the code.
2. The individual links for each team member's video recording. Each team member must create their OWN video recording (using Loom, Zoom, or similar) in which all the phases of compiler design and the working lexer are explained. Make sure to describe several theoretical principles, design practices, and implementation strategies of compiler design that can be put into practice. Then, present how the project code works, outline the individually assigned tasks that were completed, and provide a demonstration of correct execution. It is ok to have some redundancy across team members.

APA style is not required, but solid academic writing is expected.

This assignment uses a rubric. Please review the rubric prior to beginning the assignment to become familiar with the expectations for successful completion.

You are not required to submit this assignment to LopesWrite.

This assignment reinforces the following competency: Apply computer science theory and software development fundamentals to produce computing-based solutions.

Week 1 Participation

Start Date & Time	Due Date & Time	Points
Sep 8, 2026, 12:00 AM	Sep 13, 2026, 11:59 PM	10

Topic 2: Compiler for a Starter Language

Sep 14, 2026 - Oct 11, 2026 Max Points: 170

When building a compiler, it is important to not only be familiar with the theoretical principles, design practices, and implementation strategies but the various levels of complexity that can occur. Therefore, starting with a simple language (grammar) and adding features along the way is recommended.

Objectives:

1. Utilize production rules to perform syntax analysis.
2. Generate an abstract syntax tree (AST).
3. Develop a simple semantic analyzer to perform semantic checks.
4. Leverage the AST to generate intermediate code and assembly code for a given source language.

Resources

Compilers: Principles, Techniques, & Tools (REQUIRED)

Read Chapters 2–5 and browse through Chapters 6–8 in *Compilers: Principles, Techniques, & Tools*.

Assessments

Lab Question 3

Start Date & Time	Due Date & Time	Points

Start Date & Time	Due Date & Time	Points
Sep 14, 2026, 12:00 AM	Sep 16, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

Explain the difference between top-down and bottom-up parsing. Illustrate your explanation using an example of a context-free grammar.

Lab Question 4

Start Date & Time	Due Date & Time	Points
Sep 14, 2026, 12:00 AM	Sep 18, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

Explain how the syntax analyzer validates assignment statements. Refer to the appropriate grammar section and show an example, including all production rules.

Week 2 Participation

Start Date & Time	Due Date & Time	Points
Sep 14, 2026, 12:00 AM	Sep 20, 2026, 11:59 PM	10

Lab Question 5

Start Date & Time	Due Date & Time	Points
Sep 21, 2026, 12:00 AM	Sep 23, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

Explain the approach to generating meaningful and helpful compilation error messages. Write a short program that contains three deliberate syntax errors and demonstrate the correct detection of the errors and their exact location in the source code.

Lab Question 6

Start Date & Time	Due Date & Time	Points
Sep 21, 2026, 12:00 AM	Sep 25, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

Write a short program (up to 10 lines) in the language chosen as input for your compiler. Draw the AST that the syntax analyzer is expected to create. Use a drawing program.

Week 3 Participation

Start Date & Time	Due Date & Time	Points
Sep 21, 2026, 12:00 AM	Sep 27, 2026, 11:59 PM	10

Lab Question 7

Start Date & Time	Due Date & Time	Points
Sep 28, 2026, 12:00 AM	Sep 30, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

How would you generate the AST in Bison? Provide an example and include the screenshot showing the resulting AST.

Lab Question 8

Start Date & Time	Due Date & Time	Points
Sep 28, 2026, 12:00 AM	Oct 2, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

Refer to the same short program you wrote in Lab Question 6. Write the IR code that the semantic analyzer should generate.

Week 4 Participation

Start Date & Time	Due Date & Time	Points
Sep 28, 2026, 12:00 AM	Oct 4, 2026, 11:59 PM	10

Lab Question 9

Start Date & Time	Due Date & Time	Points
Oct 5, 2026, 12:00 AM	Oct 7, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

Refer to the same short program you wrote in Lab Question 8. Write the assembly code that the code generator should generate.

Lab Question 10

Start Date & Time	Due Date & Time	Points
Oct 5, 2026, 12:00 AM	Oct 9, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

Refer to the assembly code generated in Lab Question 9. Write the assembly code that the code generator should generate. Improve the code and test it in QTspim or on an ARM device. Submit a screenshot depicting execution.

CLC – Project 2: Minimalist Language Compiler

Start Date & Time	Due Date & Time	Points
Oct 5, 2026, 12:00 AM	Oct 11, 2026, 11:59 PM	90

Assessment Traits

Group

Assessment Description

This is a Collaborative Learning Community (CLC) assignment.

In this project, students will create a complete compiler for a very small language.

Complete each phase and functionality outlined within the "CST-405 Compiler Design Checklist, located in Class Resources, for the minimal grammar provided.

Then, submit the following as directed by the instructor:

1. All code to your private GitHub account making sure to provide extensive comments in the code.
2. The individual links for each team member's video recording. Each team member must create their OWN video recording (using Loom, Zoom, or similar) demonstrating the compiler working. The video should a) explain how the code works, b) outline the individually assigned tasks that were completed, c) describe the logic/process behind leveraging the AST to generate intermediate code and assembly code for a given source language, and d) demonstrate execution. It is ok to have some redundancy across team members.

APA style is not required, but solid academic writing is expected.

This assignment uses a rubric. Please review the rubric prior to beginning the assignment to become familiar with the expectations for successful completion.

You are not required to submit this assignment to LopesWrite.

This assignment reinforces the following competency: Apply computer science theory and software development

fundamentals to produce computing-based solutions.

Week 5 Participation

Start Date & Time	Due Date & Time	Points
Oct 5, 2026, 12:00 AM	Oct 11, 2026, 11:59 PM	10

Topic 3: Compiling Complex Variables and Functions

Oct 12, 2026 - Nov 1, 2026 Max Points: 220

As complexities associated with designing a programming language occur, it is important to examine expanded functionality and features that can be used. Complex variables and functions provide the tools to perform useful mathematical calculations, a fundamental use of a computer program.

Objectives:

1. Integrate increasingly complex compiler components.
2. Compile expressions involving arrays and functions.
3. Implement scope management methods.
4. Demonstrate technical interview skills within set situations.

Resources

Compilers: Principles, Techniques, & Tools (REQUIRED)

Review Chapters 2–5 and browse through Chapters 6–8 in *Compilers: Principles, Techniques, & Tools*.

Assessments

Lab Question 11

Start Date & Time	Due Date & Time	Points
Oct 12, 2026, 12:00 AM	Oct 14, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

Propose two methods for scope management and describe the pros and cons of employing each one. Include pseudocode for each method to illustrate your explanations.

Lab Question 12

Start Date & Time	Due Date & Time	Points
Oct 12, 2026, 12:00 AM	Oct 16, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

How would your parser differentiate between a variable declaration and a function declaration? Show a snippet of source code declaring a function and explain how that parser handles it.

Week 6 Participation

Start Date & Time	Due Date & Time	Points
Oct 12, 2026, 12:00 AM	Oct 18, 2026, 11:59 PM	10

Lab Question 13

Start Date & Time	Due Date & Time	Points
Oct 19, 2026, 12:00 AM	Oct 21, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

How would your parser differentiate between a variable declaration and a function declaration? Show a snippet of source code declaring a function, and the code in the parser that handles it.

Lab Question 14

Start Date & Time	Due Date & Time	Points
Oct 19, 2026, 12:00 AM	Oct 23, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

Write a short snippet of source code in the input language of your parser and include an array declaration and use. Provide the AST generated by the parser while compiling this code.

Technical Interview Quiz 1

Start Date & Time	Due Date & Time	Points
Oct 19, 2026, 12:00 AM	Oct 25, 2026, 11:59 PM	60

Assessment Description

In the industry, many positions will include a technical interview. In these, you may be required to perform highly technical skills (such as coding or solving problems) to assess your knowledge, quantitative skills, and ability to troubleshoot. Therefore, it is essential to prepare and practice in order to build confidence and ensure success.

Complete the technical interview as directed by the instructor.

Week 7 Participation

Start Date & Time	Due Date & Time	Points
Oct 19, 2026, 12:00 AM	Oct 25, 2026, 11:59 PM	10

Lab Question 15

Start Date & Time	Due Date & Time	Points
Oct 26, 2026, 12:00 AM	Oct 28, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

Write a short snippet of source code in the input language of your parser and include two simple function declarations, as well as their calls. Provide the AST generated by the parser while compiling this code. Show the content of the symbol table and demonstrate your approach to scope management.

Lab Question 16

Start Date & Time	Due Date & Time	Points
Oct 26, 2026, 12:00 AM	Oct 30, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

Write a short snippet of source code in the input language of your parser and include two simple function declarations, as well as their calls. The functions should use the same parameters, and each should define a local variable with a similar name. Provide the AST generated by the parser while compiling this code. Show the content of the symbol table and demonstrate your approach to scope management.

CLC - Project 3: Compiling Complex Variables and Functions

Start Date & Time	Due Date & Time	Points
Oct 26, 2026, 12:00 AM	Nov 1, 2026, 11:59 PM	100

Assessment Traits

Group

Assessment Description

This is a Collaborative Learning Community (CLC) assignment.

In this project, students will expand the compiler created in Project 2 to include:

1. Support for arrays
2. Support for simple mathematical expressions
3. Support for declaring and calling functions

Complete each phase and functionality outlined within the "CST-405 Compiler Design Checklist, located in Class Resources, for the expanded grammar provided.

Then, submit the following as directed by the instructor:

1. All code to your private GitHub account making sure to provide extensive comments in the code.
2. The individual links for each team member's video recording. Each team member must create their OWN video recording (using Loom, Zoom, or similar) demonstrating the compiler working. The video should a) explain how the code works, b) outline the individually assigned tasks that were completed, c) describe the team's approach to integrating increasingly complex compiler components, and d) demonstrate execution. It is ok to have some redundancy across team members.

APA style is not required, but solid academic writing is expected.

This assignment uses a rubric. Please review the rubric prior to beginning the assignment to become familiar with the expectations for successful completion.

You are not required to submit this assignment to LopesWrite.

This assignment reinforces the following competency: Apply computer science theory and software development fundamentals to produce computing-based solutions.

Week 8 Participation

Start Date & Time	Due Date & Time	Points
Oct 26, 2026, 12:00 AM	Nov 1, 2026, 11:59 PM	10

Topic 4: Compiling Loops

Nov 2, 2026 - Nov 22, 2026 Max Points: 160

Another compiler feature that expands functionality is loops. Loops provide programming language the ability to perform repetitive tasks. Often, this capability is essential for implementing iterative processes.

Objectives:

1. Expand compiler functionality to process control flow language constructs.
2. Determine language design choices to simplify compilation.
3. Optimize intermediate code.
4. Quantify the performance gain from optimization.

Resources

Compilers: Principles, Techniques, & Tools (REQUIRED)

Review Chapters 4–5, read Chapter 6, and browse through Chapters 7–8 in *Compilers: Principles, Techniques, & Tools*.

Assessments

Lab Question 17

Start Date & Time	Due Date & Time	Points
Nov 2, 2026, 12:00 AM	Nov 4, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

What production rule(s) would you need to add to your parser to recognize a *while* loop? Provide a detailed account.

Lab Question 18

Start Date & Time	Due Date & Time	Points
Nov 2, 2026, 12:00 AM	Nov 6, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

Describe some of your language design dilemmas when deciding what type of loops to implement. Use source code and compiler pseudocode to illustrate your possible approaches to implementation.

Week 9 Participation

Start Date & Time	Due Date & Time	Points
Nov 2, 2026, 12:00 AM	Nov 8, 2026, 11:59 PM	10

Lab Question 19

Start Date & Time	Due Date & Time	Points
Nov 9, 2026, 12:00 AM	Nov 11, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

How would you perform constant propagation optimization? Use source code and compiler pseudocode to illustrate your possible approaches to implementation.

Lab Question 20

Start Date & Time	Due Date & Time	Points
Nov 9, 2026, 12:00 AM	Nov 13, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

What is dead code elimination? Illustrate the concept with appropriate pseudocode and explain the benefits, if any, in terms of resource utilization. Show the unoptimized and optimized intermediate code.

Week 10 Participation

Start Date & Time	Due Date & Time	Points
Nov 9, 2026, 12:00 AM	Nov 15, 2026, 11:59 PM	10

Lab Question 21

Start Date & Time	Due Date & Time	Points
Nov 16, 2026, 12:00 AM	Nov 18, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

What is loop unfolding? Illustrate the concept with appropriate pseudocode and explain the benefits, if any, in terms of resource utilization. Show the unoptimized and optimized intermediate code.

Lab Question 22

Start Date & Time	Due Date & Time	Points
Nov 16, 2026, 12:00 AM	Nov 20, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

How would you measure the performance of an optimized compiler? Using the specifications of your computer as reference, what would you consider a significant performance gain? Provide specific quantitative information to support your answer.

CLC - Project 4: Updated Compiler to Support Loops

Start Date & Time	Due Date & Time	Points
Nov 16, 2026, 12:00 AM	Nov 22, 2026, 11:59 PM	100

Assessment Traits

Group

Assessment Description

This is a Collaborative Learning Community (CLC) assignment.

In this project, students will expand the compiler created in Project 3 to include:

1. Support for at least one type of loop (e.g., while, for, or repeat)
2. Support for order of operations

Complete each phase and functionality outlined within the "CST-405 Compiler Design Checklist," located in Class Resources, for the expanded grammar provided.

Then, submit the following as directed by the instructor:

1. All code to your private GitHub account making sure to provide extensive comments in the code.
2. The individual links for each team member's video recording. Each team member must create their OWN video recording (using Loom, Zoom, or similar) demonstrating the compiler working. The video should a) explain how the code works, b) outline the individually assigned tasks that were completed, c) describe the language design choices made, and d) demonstrate execution. It is ok to have some redundancy across team members.

APA style is not required, but solid academic writing is expected.

This assignment uses a rubric. Please review the rubric prior to beginning the assignment to become familiar with the expectations for successful completion.

You are not required to submit this assignment to LopesWrite.

This assignment reinforces the following competency: Apply computer science theory and software development fundamentals to produce computing-based solutions.

Week 11 Participation

Start Date & Time	Due Date & Time	Points
Nov 16, 2026, 12:00 AM	Nov 22, 2026, 11:59 PM	10

Topic 5: Compiling Control Flow – Decisions (e.g., IF Statements)

Nov 23, 2026 - Dec 13, 2026 Max Points: 160

In expanding a compiler, it is important to understand how decisions are made by the program. This is known as control flow. Control flow allows for the execution of code in a specific order.

Objectives:

1. Demonstrate the ability to expand a compiler in a manner that increases the complexity of the language structures it can handle.
2. Audit the correctness of the AST and semantic actions of the parser.
3. Improve and broaden the IR code optimization capabilities of a compiler.
4. Compare code generation for MIPS vs. ARM processors.

Resources

Compilers: Principles, Techniques, & Tools (REQUIRED)

Review Chapters 4–6 and read Chapters 7–9 in *Compilers: Principles, Techniques, & Tools*.

Assessments

Lab Question 23

Start Date & Time	Due Date & Time	Points
Nov 23, 2026, 12:00 AM	Nov 25, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

What production rule(s) would you need to add to your parser to recognize an *if* statement and an *if-then-else* construct? Provide a detailed account.

Lab Question 24

Start Date & Time	Due Date & Time	Points
Nov 23, 2026, 12:00 AM	Nov 27, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

Describe some of your language design dilemmas when deciding to implement features associated with logical decisions. Use source code and compiler pseudocode to illustrate your possible approaches to implementation.

Week 12 Participation

Start Date & Time	Due Date & Time	Points
Nov 23, 2026, 12:00 AM	Nov 29, 2026, 11:59 PM	10

Lab Question 25

Start Date & Time	Due Date & Time	Points
Nov 30, 2026, 12:00 AM	Dec 2, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

What is the *dangling else* problem? Illustrate the concept with appropriate pseudo-code and explain how you

address it in your compiler.

Lab Question 26

Start Date & Time	Due Date & Time	Points
Nov 30, 2026, 12:00 AM	Dec 4, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

How would you implement nested *if* statements in your compiler (parser, semantic analyzer, code generator)? Provide detailed steps.

Week 13 Participation

Start Date & Time	Due Date & Time	Points
Nov 30, 2026, 12:00 AM	Dec 6, 2026, 11:59 PM	10

Lab Question 27

Start Date & Time	Due Date & Time	Points
Dec 7, 2026, 12:00 AM	Dec 9, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

Write a short snippet of source code in the input language of your compiler and include two simple function declarations, as well as their calls. One function should include an *if* statement. The second function should include a loop. Provide the AST generated by the parser while compiling this code. Show the content of the symbol table and demonstrate your approach to scope management.

Lab Question 28

Start Date & Time	Due Date & Time	Points
Dec 7, 2026, 12:00 AM	Dec 11, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

Using the example and output in Lab Question 27, show the unoptimized intermediate code generated by your compiler, at least three optimizations, the optimized intermediate code, and the assembly code. Execute the assembly code and provide screenshots depicting correct execution.

CLC – Project 5: Updated Compiler to Support Logic Decisions

Start Date & Time	Due Date & Time	Points
Dec 7, 2026, 12:00 AM	Dec 13, 2026, 11:59 PM	100

Assessment Traits

Group

Assessment Description

This is a Collaborative Learning Community (CLC) assignment.

In this project, students will expand the compiler created in project 4 to include:

1. Support for decision controls: if-then, if-then-else, nested if statements (optional switch)
2. Support for logical (Boolean) operations

Complete each phase and functionality outlined within the "CST-405 Compiler Design Checklist," located in Class Resources, for the expanded grammar provided.

Then, submit the following as directed by the instructor:

1. All code to your private GitHub account making sure to provide extensive comments in the code.
2. The individual links for each team member's video recording. Each team member must create their OWN video recording (using Loom, Zoom, or similar) demonstrating the compiler working. The video should a) explain how the code works, b) outline the individually assigned tasks that were completed, c) describe how to increase the complexity of language structures a compiler can handle, and d) demonstrate execution. It is ok to have some redundancy across team members

APA style is not required, but solid academic writing is expected.

This assignment uses a rubric. Please review the rubric prior to beginning the assignment to become familiar with the expectations for successful completion.

You are not required to submit this assignment to LopesWrite.

This assignment reinforces the following competency: Apply computer science theory and software development fundamentals to produce computing-based solutions.

Week 14 Participation

Start Date & Time	Due Date & Time	Points
Dec 7, 2026, 12:00 AM	Dec 13, 2026, 11:59 PM	10

Topic 6: Compiler Design and Implementation

Dec 14, 2026 - Dec 20, 2026 Max Points: 180

Upon completion of any compiler, it is essential to review the concepts and tools used in its development. By analyzing this information, as well as the performance results, one can gauge where any future issues or concerns may present themselves.

Objectives:

1. Demonstrate a fully working compiler.
2. Demonstrate handling of syntax and semantic errors.
3. Analyze compiler performance.
4. Analyze executable code efficiency.
5. Demonstrate team collaboration skills in meeting project goals.
6. Communicate project planning and implementation to various audiences.

Resources

Compilers: Principles, Techniques, & Tools (REQUIRED)

Review Chapters 4–9 in *Compilers: Principles, Techniques, & Tools*.

Assessments

Lab Question 29

Start Date & Time	Due Date & Time	Points
Dec 14, 2026, 12:00 AM	Dec 16, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

Reflect on the entire compiler project. Discuss three things you would have done differently given your current knowledge. Of the three things, one should relate to the compiler design, the second to features of the language, and the third to the code implementation.

Lab Question 30

Start Date & Time	Due Date & Time	Points
Dec 14, 2026, 12:00 AM	Dec 18, 2026, 11:59 PM	5

Assessment Traits

Accommodated

Assessment Description

Describe three features of your compiler that you are most proud of. Include relevant code snippets and screenshots depicting these features.

Benchmark – Project 6: Complete Compiler

Start Date & Time	Due Date & Time	Points
Dec 14, 2026, 12:00 AM	Dec 20, 2026, 11:59 PM	100

Assessment Traits

Accommodated

Benchmark

Assessment Description

Although this project will be completed as a group assignment, it will be individually submitted by each student.

In this project, students will update the compiler created in Assignment 5 to include:

1. Comprehensive README file with installation and usage instructions
2. Performance metrics, to include a) compilation time and b) execution time of compiler code

Then, submit the following as directed by the instructor:

1. All code to your private GitHub account making sure to provide extensive comments in the code.
2. The individual links for each team member's video recording. Each team member must create their OWN video recording (using Loom, Zoom, or similar) demonstrating the compiler working and explaining decisions made regarding a) language design and b) your approach to implementation.

APA style is not required, but solid academic writing is expected.

This assignment uses a rubric. Please review the rubric prior to beginning the assignment to become familiar with the expectations for successful completion.

You are not required to submit this assignment to LopesWrite.

Benchmark Information

This benchmark assignment assesses the following programmatic competencies:

BS-Computer Science with an Emphasis in Big Data Analytics

BS-Computer Science with an Emphasis in Business Entrepreneurship

BS-Computer Science with an Emphasis in Game and Simulation Development

4.1 Develop a program using assembly language that provides a solution to an architectural challenge.

4.2 Develop software that makes efficient use of system resources.

4.3 Communicate effectively and convey professional demeanor in a variety of professional contexts (ABET 3).

6.1 Select and utilize data structures appropriate to a given computational problem.

7.4 Function effectively as a member or leader of a team engaged in activities appropriate to the program's discipline (ABET 5)

Technical Interview Quiz 2

Start Date & Time	Due Date & Time	Points
Dec 14, 2026, 12:00 AM	Dec 20, 2026, 11:59 PM	60

Assessment Description

In the industry, many positions will include a technical interview. In these, you may be required to perform highly technical skills (such as coding or solving problems) to assess your knowledge, quantitative skills, and

ability to troubleshoot. Therefore, it is essential to prepare and practice in order to build confidence and ensure success.

Complete the technical interview as directed by the instructor.

Week 15 Participation

Start Date & Time	Due Date & Time	Points
Dec 14, 2026, 12:00 AM	Dec 20, 2026, 11:59 PM	10